Chapter Three

Mathematical analysis of both recursive and non-recursive algorithm

Outline of the chapter

3.1 Introduction

3.2 Complexity bounds, Probabilistic analysis and randomized algorithms

3.3 Amortized analysis with examples of amortization

3.4 Master's theorem and its examples

3.5 Examples of non-recursive algorithms

3.6 Examples of recursive algorithms

3.1 Introduction

[N.B this slide presented as revision or recall]

- **Probability** is an essential tool for the design and analysis of probabilistic and randomized algorithms. probability define in terms of a ***sample space*** S, which is a set whose elements are called ***elementary events***.
- For the experiment of flipping two distinguishable coins, we can view the sample space as consisting of the set of all possible
  - 2-strings over {H, T}: S = {HH, HT, TH, TT} .
- An ***event*** is a subset1 of the sample space S.
  - For example, in the experiment of flipping two coins, the event of obtaining one head and one tail is {HT, TH}.
- The event S is called the ***certain event***, and the event $\emptyset$ is called the ***null event***. We say that two events A and B are ***mutually exclusive*** if $A \cap B = \emptyset$. We sometimes treat an elementary event $s \in S$ as the event $\{s\}$.
- By definition, all elementary events are mutually exclusive.

3.2 Complexity bounds, Probabilistic analysis and randomized algorithms

Complexity bound

Worst-Case Running Time

- We are interested on how the running time **scales** with input size
- Normally we are interested in **worst case** running time (worst case of all inputs of a given size)

Definition - Worst-Case Running Time

I - some input
T(**I**) - running time for input *I*
T(**n**) - worst-case running time for input size *n*

$$T(n) = \max_{\text{inputs } I \text{ of size } n}\{T(I)\}$$

Average-Case Running Time

- We could be interested in **average-case**
- Measure performance in "**typical**" inputs
  - What is a typical input?
- There are algorithms with **large** gap between "average performance" and worst case.
- Can we **improve** worst-case by adding randomization?

The hiring problem

- Imagine you need to hire a new office assistant
- Suppose you use the following algorithm:

Hiring Algorithm for *n* candidates

```
best = 0 // candidate 0 is a least-qualified dummy candidate
for i = 1 to n
    interview candidate i
    if candidate i is better than candidate best best = i
      hire candidate i
```

- We will focus not on the running time, but instead on the **costs incurred by interviewing and hiring**.
  - Interviewing has a low cost (let's call it c_i)
  - Hiring has an higher cost (let's call it c_h)

N.B for detail of Hiring Problem refer the note part of this slide

The hiring problem: cost

- What is the cost of the algorithm?
- Suppose you end up hiring m persons out of the n candidates
- The total cost is $O(c_i n + c_h m)$:
  - No matter how many people we hire, we always interview n candidates
  - The cost associated with interviewing is always used: $c_i n$
  - We can therefore focus on the hiring cost $(c_h m)$, which varies according to the candidates and the order in which they are interviewed
- Note how this models a common algorithm paradigm:
  - finding a min or max value in a sequence and how often we update the notion on who's winning

The hiring problem: cost

- Assume we have a **total order** among all the candidates
  - We can compare two candidates and decide which one is better
  - We can rank each candidate with an unique number from 1 to n.
    - We will use $rank(i)$ to denote the rank of candidate i

    (assume an higher rank means a better candidate)
- The input can now be described as a permutation of $\langle 1, 2, \ldots, n \rangle$

- **Worst-case analysis**
  - What is the worst possible input?
  - If the candidates come in **increasing ranking order**, then we will need to hire all of them: $O(c_h n)$
  - Good thing this is not always the case... In fact we don't know in which order will they come. What would happen on a typical case? (what would the best-case scenario be?)

Probabilistic Analysis

Probabilistic Analysis

The use of **probability** in the analysis of algorithms:

- We must have some knowledge or make assumptions about the **distribution of the input**
- We can then make an **average-case analysis**, averaging the cost over all possible inputs
- For our problem, we could assume all possible permutations are **equally likely** (the ranks form an **uniform random permutation**).
  - But what if the above is not the case? What if the real distribution is skewed and some permutations are more likely than other?

Hiring Problem: a randomized algorithm

Randomized Hiring Algorithm for *n* candidates

```
randomly permute the list of candidates // the only "new" line
best = 0 // candidate 0 is a least-qualified dummy candidate
for i = 1 to n
    interview candidate i
    if candidate i is better than candidate best best = i
      hire candidate i
```

- In order to have **great control over the order** of the candidates, we could explicitly choose **randomly** which candidate to interview next
  - We now are enforcing a **random order**, regardless of the input!

Randomized Algorithms

Randomized algorithms

We call an algorithm **randomized** if its behavior is determined not only by its input but also by values produced by a **random-number generator**

- Most programming environments offer a (deterministic) **pseudorandom-number generator**: it returns numbers that *"look"* statistically random
- We typically refer to the analysis of randomized algorithms by talking about the **expected cost** (ex: the **expected running time**)
- We can use **probabilistic analysis** to analyse randomized algorithms

Basics of Probabilistic Analysis

- Consider rolling **two dice** and observing the results.
- We call this an **experiment**.
- It has **36 possible outcomes**:

  1-1, 1-2, 1-3, 1-4, 1-5, 1-6, 2-1, 2-2, 2-3, ..., 6-4, 6-5, 6-6
- Each of these outcomes has probability **1/36** (assuming fair dice)
- What is the probability of the sum of dice being 7?

  Add the probabilities of all the outcomes satisfying this condition:
  1-6, 2-5, 3-4, 4-3, 5-2, 6-1 (probability is **1/6**)

Basics of Probabilistic Analysis

In the language of probability theory, this setting is characterized by a **sample space** S and a **probability measure** p.

- **Sample Space** is constituted by all possible outcomes, which are called **elementary events**
- In a **discrete probability distribution** (d.p.d.), the probability measure is a function $p(e)$ (or $Pr(e)$) over elementary events e such that:
  - $p(e) \geq 0$ for all $e \in S$
  - $\sum_{e \in S} p(e) = 1$
- An **event** is a subset of the sample space.
- For a d.p.d. the probability of an event is just the **sum** of the probabilities of its elementary events.

Basics of Probabilistic Analysis

- A **random variable** is a function from elementary events to integers or reals:

  Ex: let X_1 be a random variable representing result of first die and X_2 representing the second die.
  $X = X_1 + X_2$ would represent the sum of the two
  We could now ask: what is the probability that $X = 7$?

- One property of a random variable we care is **expectation**:

Expectation

For a discrete random variable X over sample space S, the expected value of X is:

$$\mathbf{E}[X] = \sum_{e \in S} Pr(e)X(e)$$

Basics of Probabilistic Analysis

- In **words**: the expectation of a random variable X is just its average value over S, where each elementary event e is weighted according to its probability.

  Ex: If we roll a single die, the expected value is 3.5 (all six elementary events have equal probability).

- One possible rewrite of the previous equation, grouping elementary events:

Expectation (possible rewrite)

$$\mathbf{E}[X] = \sum_a Pr(X = a)a$$

Basics of Probabilistic Analysis

- More generally:

Expectation (rewrite using disjoint events)

For any partition of the sample space into disjoint events $A_1, A_2, \ldots$:

$$\mathbf{E}[X] = \sum_i \sum_{e \in A_i} Pr(e)X(e) = \sum_i Pr(A_i)\,\mathbf{E}[X|A_i]$$

- $\mathbf{E}[X|A_i]$ is the expected value of X given A_i, defined to be: $\frac{1}{Pr(A_i)} \sum_{e \in A_i} Pr(e)X(e)$.

Basics of Probabilistic Analysis

An important fact about expected values is **Linearity of Expectation**:

Theorem - Linearity of Expectation

For any two random variables X and Y: $\mathbf{E}[\mathbf{X}+\mathbf{Y}] = \mathbf{E}[\mathbf{X}] + \mathbf{E}[\mathbf{Y}]$

Proof for discrete random variables:

$$\mathbf{E}[X+Y] = \sum_{e \in S} Pr(e)(X(e)+Y(e)) =$$

$$= \sum_{e \in S} Pr(e)X(e) + \sum_{e \in S} Pr(e)Y(e) = \mathbf{E}[X] + \mathbf{E}[Y]$$

- It is not necessary that the variables are independent
- This theorem is **very important for the analysis of algorithms**: complicated variables become a sum of simple variables which we can analyse separately.

Hiring Problem: probabilistic analysis

- Let's use the following random indicator variable:
  Consider the event the i-th iteration of the loop in the algorithm:
  $$X_i = \begin{cases} 1 & \text{if candidate } i \text{ is hired} \\ 0 & \text{otherwise} \end{cases}$$
- $\mathbf{E}[X_i] = Pr(\text{candidate } i \text{ is hired})$
- Because the order is random:
  - Cand. i has prob. $1/i$ of being better than cand. 1 through $i-1$ (can you see why?)
  - $\mathbf{E}[X_i] = 1/i$
- Let X be the **number of candidates we hire**. Then:
  $X = X_1 + X_2 + \ldots + X_n$
  $$\mathbf{E}[X] = \mathbf{E}[\sum_{i=1}^{n} X_i] = \sum_{i=1}^{n} \mathbf{E}[X_i] = \sum_{i=1}^{n} 1/i = \ln(n) + \mathcal{O}(1)$$
  [the sum $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ is known as the **harmonic series** and $H_n = \ln(n) + \mathcal{O}(1)$]
- We interview n people, but we only hire approximately $\ln(n)$ of them!
- The **expected hiring cost** is $\mathcal{O}(c_h \log n)$

3.3 Amortized analysis with examples of amortization

- Not just consider one operation, but a sequence of operations on a given data structure.
- Average cost over a sequence of operations.
- Probabilistic analysis:
  - Average case running time: average over all possible inputs for one algorithm (operation).
  - If using probability, called expected running time.
- Amortized analysis:
  - No involvement of probability
  - Average performance on a sequence of operations, even some operation is expensive.
  - Guarantee average performance of each operation among the sequence in worst case.

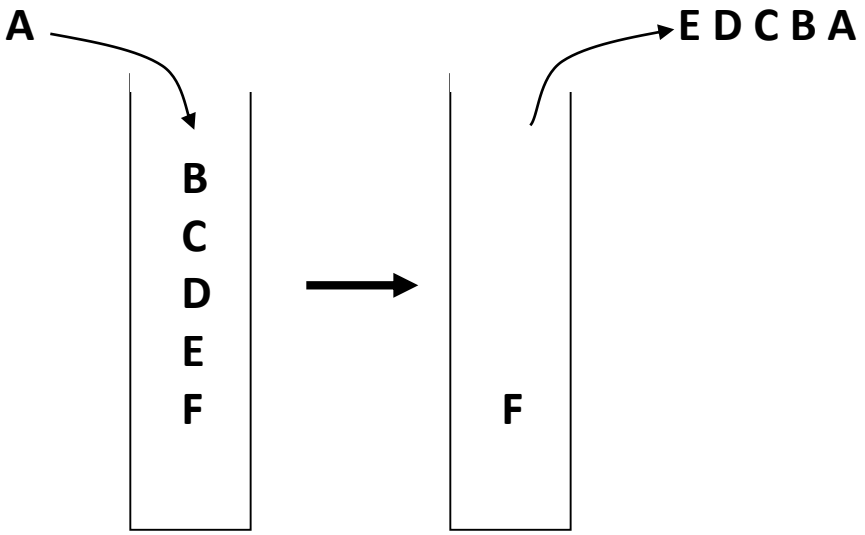
Amortized Analysis cont'd

- Consider any sequence of operations applied to a data structure
  - *your worst enemy could choose the sequence!*
- Some operations may be fast, others slow
- Goal: show that the average time per operation is still good

$$\frac{\text{total time for n operations}}{n}$$

Amortized Analysis Stack ADT

- Stack operations
  - push
  - pop
  - is_empty



- Stack property: if x is on the stack before y is pushed, then x will be popped after y is popped

Three Methods of Amortized Analysis

- Aggregate analysis:
  - Total cost of *n* operations/*n*,
- Accounting method:
  - Assign each type of operation an (different) amortized cost
  - overcharge some operations,
  - store the overcharge as credit on specific objects,
  - then use the credit for compensation for some later operations.
- Potential method: (will not cover in this Chapter)
  - Same as accounting method
  - But store the credit as "potential energy" and as a whole.

Example for amortized analysis

- Stack operations:
  - PUSH(S,x), $O(1)$
  - POP(S), $O(1)$
  - MULTIPOP(S,k), min(s,k)
    - while not STACK-EMPTY(S) and k>0
    - do POP(S)
    - k=k-1
- Let us consider a sequence of n PUSH, POP, MULTIPOP.
  - The worst case cost for MULTIPOP in the sequence is $O(n)$, since the stack size is at most n.
  - thus the cost of the sequence is $O(n^2)$. Correct, but not tight.

Aggregate Analysis

- In fact, a sequence of n operations on an initially empty stack cost at most $O(n)$. Why?
- Each object can be POP only once (including in MULTIPOP) for each time it is PUSHed. #POPs is at most #PUSHs, which is at most n.
- Thus the average cost of an operation is $O(n)/n = O(1)$.
- Amortized cost in aggregate analysis is defined to be average cost.

Another example: increasing a binary counter

- *Binary counter of length k, A[0..k-1] of bit array.*
- *INCREMENT(A)*

```
i ← 0
while i<k and A[i]=1
        do A[i] ← 0 (flip, reset)
           i ← i+1
if i<k
then A[i] ← 1          (flip, set)
```

Analysis of INCREMENT(A)

- Cursory analysis:
  - A single execution of INCREMENT takes $O(k)$ in the worst case (when A contains all 1s)
  - So a sequence of n executions takes $O(nk)$ in worst case (suppose initial counter is 0).
  - This bound is correct, but not tight.
- The tight bound is $O(n)$ for n executions.

Amortized (Aggregate) Analysis of INCREMENT(A)

Observation: **The running time determined by #flips but not all bits flip each time INCREMENT is called.**

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figure 17.2 An 8-bit binary counter as its value goes from 0 to 16 by a sequence of 16 INCREMENT operations. Bits that flip to achieve the next value are shaded. The running cost for flipping bits is shown at the right. Notice that the total cost is never more than twice the total number of INCREMENT operations.

A[0] flips every time, total n times.
A[1] flips every other time, $\lfloor n/2 \rfloor$ times.
A[2] flips every forth time, $\lfloor n/4 \rfloor$ times.
….
for i=0,1,…,k-1, A[i] flips $\lfloor n/2^i \rfloor$ times.

Thus total #flips is $\sum_{i=0}^{k-1} \lfloor n/2^i \rfloor$

$$< n\sum_{i=0}^{\infty} 1/2^i$$

$$=2n.$$

Amortized Analysis of INCREMENT(A)

- Thus the worst case running time is $O(n)$ for a sequence of n INCREMENTs.
- So the amortized cost per operation is $O(1)$.

Amortized Analysis: Accounting Method

- Idea:
  - Assign differing charges to different operations.
  - The amount of the charge is called amortized cost.
  - amortized cost is more or less than actual cost.
  - When amortized cost > actual cost, the difference is saved in specific objects as credits.
  - The credits can be used by later operations whose amortized cost < actual cost.
- As a comparison, in aggregate analysis, all operations have same amortized costs.

Accounting Method (cont.)

- Conditions:
  - suppose actual cost is c_i for the ith operation in the sequence, and amortized cost is c_i',
  - $\sum_{i=1}^{n} c_i' \geq \sum_{i=1}^{n} c_i$ should hold.
    - since we want to show the average cost per operation is small using amortized cost, we need the total amortized cost is an upper bound of total actual cost.
    - holds for all sequences of operations.
  - Total credits is $\sum_{i=1}^{n} c_i' - \sum_{i=1}^{n} c_i$, which should be nonnegative,
    - Moreover, $\sum_{i=1}^{t} c_i' - \sum_{i=1}^{t} c_i \geq 0$ for any $t>0$.

Accounting Method: Stack Operations

- Actual costs:
  - PUSH :1, POP :1, MULTIPOP: min(s,k).
- Let assign the following amortized costs:
  - PUSH:2, POP: 0, MULTIPOP: 0.
- Similar to a stack of plates in a cafeteria.
  - Suppose $1 represents a unit cost.
  - When pushing a plate, use one dollar to pay the actual cost of the push and leave one dollar on the plate as credit.
  - Whenever POPing a plate, the one dollar on the plate is used to pay the actual cost of the POP. (same for MULTIPOP).
  - By charging PUSH a little more, do not charge POP or MULTIPOP.
- The total amortized cost for n PUSH, POP, MULTIPOP is $O(n)$, thus $O(1)$ for average amortized cost for each operation.
- Conditions hold: total amortized cost ≥total actual cost, and amount of credits never becomes negative.

Accounting method: binary counter

- Let \$1 represent each unit of cost (i.e., the flip of one bit).
- Charge an amortized cost of \$2 to set a bit to 1.
- Whenever a bit is set, use \$1 to pay the actual cost, and store another \$1 on the bit as credit.
- When a bit is reset, the stored \$1 pays the cost.
- At any point, a 1 in the counter stores \$1, the number of 1's is never negative, so is the total credits.
- At most one bit is set in each operation, so the amortized cost of an operation is at most \$2.
- Thus, total amortized cost of n operations is $O(n)$, and average is $O(1)$.

The Potential Method

- Same as accounting method: something prepaid is used later.
- Different from accounting method
  - The prepaid work not as credit, but as “potential energy”, or “potential”.
  - The potential is associated with the data structure as a whole rather than with specific objects within the data structure.

3.4 Master's theorem and its examples

- The master method provides a "cookbook" method for solving recurrences of the form
  - $T(n) = aT(n/b) + f(n)$, ... (4.5)
  - where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function.
- The master method requires memorization of three cases, but then the solution of many recurrences can be determined quite easily, often without pencil and paper.
- The recurrence (4.5) describes the running time of an algorithm that divides a problem of size n into a subproblems, each of size n/b, where a and b are positive constants.
- The a subproblems are solved recursively, each in time T (n/b).
- The cost of dividing the problem and combining the results of the subproblems is described by the function f (n).
  - $f(n) = D(n)+C(n)$.)
  - For example, the recurrence arising from the MERGE-SORT procedure has $a = 2$, $b = 2$, and $f(n) = (n)$.

3.4 Master's theorem cont'd

- As a matter of technical correctness, the recurrence isn't actually well defined because n/b might not be an integer.
- Replacing each of the a terms T (n/b) with either T (n/b) or T (n/b) doesn't affect the asymptotic behavior of the recurrence,
- therefore, to omit the floor and ceiling functions when writing divide-and conquer recurrences of this form.

3.4 Master's theorem cont'd

- **The master method depends on the following theorem.** Theorem 4.1 (Master theorem)
  - Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ can be bounded asymptotically as follows.

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

3.4 Master's theorem cont'd

- In each of the three cases, we are comparing the function *f (n)* with the function $n^{\log_b a}$
- Intuitively, the solution to the recurrence is determined by the larger of the two functions.
- If, as in case 1, the function $n^{\log_b a}$ is
- the larger, then the solution is *T (n)* = $\Theta(n^{\log_b a})$
- If, as in case 3, the function *f (n)* is the larger, then the solution is *T (n)* = $\Theta(f(n))$
- If, as in case 2, the two functions are the same size, we multiply by a logarithmic factor, and the solution is

$$T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n).$$

3.4 Master's theorem cont'd

- Beyond this intuition, there are some technicalities that must be understood.
- In the first case, not only must f (n) be smaller than $n^{\log_b a}$, it must be polynomially smaller.
- That is, f (n) must be asvmptotically smaller than $n^{\log_b a}$ by a factor n^{ϵ} of for some constant $\epsilon > 0$. In the third case, not only must f (n) be larger than , $n^{\log_b a}$

  it must be polynomially larger and in addition satisfy the "regularity" condition that a f (n/b) ≤ cf (n).
- This condition is satisfied by most of the polynomially bounded functions that we shall encounter.

3.4 Master's theorem cont'd

- It is important to realize that the three cases do not cover all the possibilities for f (n).
  - There is a gap between cases 1 and 2 when f (n) is smaller than $n^{\log_b a}$ but not polynomially smaller.
  - Similarly, there is a gap between cases 2 and 3 when f (n) is larger than $n^{\log_b a}$ but not polynomially larger.
  - If the function f (n) falls into one of these gaps, or if the regularity condition in case 3 fails to hold, the master method cannot be used to solve the recurrence.

3.4 Master's theorem cont'd

Using the master method

$T(n) = 9T(n/3) + n$.

For this recurrence, we have $a = 9$, $b = 3$, $f(n) = n$, and thus we have that $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \epsilon})$, where $\epsilon = 1$, we can apply case 1 of the master theorem and conclude that the solution is $T(n) = \Theta(n^2)$.

Now consider

$T(n) = T(2n/3) + 1$,

in which $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Case 2 applies, since $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, and thus the solution to the recurrence is $T(n) = \Theta(\lg n)$.

Summarized Understanding the Master Theorem

MASTER THEOREM

$$T(n) = \begin{cases} d & \text{if } n \text{ is at most some constant} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{otherwise} \end{cases}$$

Where $f(n)$ is $\Theta(n^c)$

If $\log_b a < c$ then $T(n) \in \Theta(n^c)$

If $\log_b a = c$ then $T(n) \in \Theta(n^c \log n)$

If $\log_b a > c$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$

- A measures how many recursive calls are triggered by each method instance
- B measures the rate of change for input
- C measures the dominating term of the non recursive work within the recursive method
- D measures the work done in the base case

- The $\log_b a < c$ case
  - Recursive case does a lot of non recursive work in comparison to how quickly it divides the input size
  - Most work happens in beginning of call stack
  - Non recursive work in recursive case dominates growth, n^c term
- The $\log_b a = c$ case
  - Recursive case evenly splits work between non recursive work and passing along inputs to subsequent recursive calls
  - Work is distributed across call stack
- The $\log_b a > c$ case
  - Recursive case breaks inputs apart quickly and doesn't do much non recursive work
  - Most work happens near bottom of call stack

3.5 Examples of non-recursive algorithms

General Plan for Analyzing the Time Efficiency of Non recursive Algorithms:

1. Decide on a *parameter* (or parameters) indicating an input's size.
2. Identify the algorithm's *basic operation* (in the inner most oop).
3. Check whether the *number of times the basic operation is executed* depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case *efficiencies* have to be investigated separately.
4. Set up a *sum* expressing the number of times the algorithm's basic operation is executed.
5. Using standard formulas and rules of sum manipulation either find a closed form formula for the count or at the least, establish its *order of growth*.

3.5 Examples of non-recursive algorithms

Example 1: Maximum element

ALGORITHM *MaxElement*$(A[0..n-1])$

```
//Determines the value of the largest element in a given array
//Input: An array A[0..n − 1] of real numbers
//Output: The value of the largest element in A
maxval ← A[0]
for i ← 1 to n − 1 do
    if A[i] > maxval
        maxval ← A[i]
return maxval
```

3.5 Examples of non-recursive algorithms

Example 2: Element uniqueness problem

ALGORITHM *UniqueElements*$(A[0..n-1])$

```
//Determines whether all the elements in a given array are distinct
//Input: An array A[0..n − 1]
//Output: Returns "true" if all the elements in A are distinct
//        and "false" otherwise
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i] = A[j] return false
return true
```

3.5 Examples of non-recursive algorithms

Example 3: Matrix multiplication

ALGORITHM *MatrixMultiplication*$(A[0..n-1, 0..n-1], B[0..n-1, 0..n-1])$

```
//Multiplies two n-by-n matrices by the definition-based algorithm
//Input: Two n-by-n matrices A and B
//Output: Matrix C = AB
for i ← 0 to n − 1 do
    for j ← 0 to n − 1 do
        C[i, j] ← 0.0
        for k ← 0 to n − 1 do
            C[i, j] ← C[i, j] + A[i, k] * B[k, j]
return C
```

3.5 Examples of non-recursive algorithms

Example 4: Gaussian elimination

Algorithm *GaussianElimination*($A[0..n-1,0..n]$)

//Implements Gaussian elimination of an n-by-(n+1) matrix A

for $i \leftarrow$ 0 to n - 2 do

  for $j \leftarrow$ i + 1 to n - 1 do

  for $k \leftarrow n$ downto i do

    $A[j,k] \leftarrow A[j,k] - A[i,k] * A[j,i] / A[i,i]$

Find the efficiency class and a constant factor improvement.

3.6 Examples of recursive algorithms

Recursion

- A recursive procedure can often be analyzed by solving a recursive equation
- Basic form:

  T(n) = if (base case) then some constant
  else (time to solve subproblems +
  time to combine solutions)

- Result depends upon
  - how many subproblems
  - how much smaller are subproblems
  - how costly to combine solutions (coefficients)

Example: Sum of Integer Queue

```
sum_queue(Q){
  if (Q.length == 0 ) return 0;
  else return Q.dequeue() +
                 sum_queue(Q);  }
```

- One subproblem
- Linear reduction in size (decrease by 1)
- Combining: constant c (+), 1×subproblem

Equation: $T(0) \leq b$

$T(n) \leq c + T(n-1)$ for $n>0$

Sum, Continued

Equation: $T(0) \leq b$

$T(n) \leq c + T(n-1)$ for $n>0$

Solution:

$$
\begin{aligned}
T(n) &\leq c + c + T(n-2) \\
&\leq c + c + c + T(n-3) \\
&\leq kc + T(n-k) \quad \text{for all } k \\
&\leq nc + T(0) \quad \text{for } k=n \\
&\leq cn + b \quad = \quad O(n)
\end{aligned}
$$

Example: Binary Search

7	12	30	35	75	83	87	90	97	99

One subproblem, half as large

Equation: $T(1) \le b$

$T(n) \le T(n/2) + c$ for $n>1$

Solution:

$$
\begin{aligned}
T(n) &\le T(n/2) + c \\
&\le T(n/4) + c + c \\
&\le T(n/8) + c + c + c \\
&\le T(n/2^k) + kc \\
&\le T(1) + c \log n \quad \text{where } k = \log n \\
&\le b + c \log n \quad = \quad O(\log n)
\end{aligned}
$$

Example: MergeSort

Split array in half, sort each half, merge together

- 2 subproblems, each half as large
- linear amount of work to combine

$$T(1) \le b$$

$$T(n) \le 2T(n/2) + cn \qquad \text{for } n>1$$

$$T(n) \le 2T(n/2)+cn \quad \le \quad 2(2(T(n/4)+cn/2)+cn$$

$$= 4T(n/4) +cn +cn \quad \le \quad 4(2(T(n/8)+c(n/4))+cn+cn$$

$$= 8T(n/8)+cn+cn+cn \quad \le \quad 2kT(n/2k)+kcn$$

$$\le 2kT(1) + cn \log n \qquad \text{where } k = \log n$$

$$= O(n \log n)$$